Modele regresji

Statystyka w SI i analizie danych — laboratorium 5

[Wojciech Maćkowiak, 412430]

2026-04-12

Contents

1 Wstęp — definicje miar i testów **1**

2 Zadanie 1 — Regresja liniowa i interakcje **2**
2.1 1.1 Real Estate Valuation (UCI) 2
2.2 1.2 Auto MPG (UCI; zmienna zależna: `mpg`) 7

3 Zadanie 2 — Regresja Poissona (`poisson.data`) **11**

4 Zadanie 3 — `ships` (MASS) **12**

5 Zadanie 4 — `esoph` **14**

6 Zadanie 5 — CLM (`clm.data`) **16**

7 Zadanie 6 — `wine` (ordinal) **17**

8 Podsumowanie **19**

1 Wstęp — definicje miar i testów

W całym sprawozdaniu stosujemy następujące definicje.

Współczynnik determinacji $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ — udział wariancji objaśnianej przez model.

VIF (Variance Inflation Factor) dla predyktora X_j: $\mathrm{VIF}_j = \frac{1}{1-R_j^2}$, gdzie R_j^2 pochodzi z regresji X_j na pozostałe predyktory. Wartości $\gg$ 5–10 sugerują współliniowość.

Test Breuscha–Pagana (homoskedastyczność) — H_0: wariancja reszt jest stała; statystyka oparta na regresji $\hat{\varepsilon}_i^2$ na predyktorach.

Test Durbin–Watson (autokorelacja) — statystyka $DW = \frac{\sum_{t=2}^{n} (\hat{\varepsilon}_t - \hat{\varepsilon}_{t-1})^2}{\sum_{t=1}^{n} \hat{\varepsilon}_t^2}$; dla braku autokorelacji $DW \approx 2$.

Deviance w GLM: $D = 2(\ell_{\text{sat}} - \ell_{\text{model}})$, gdzie ℓ to log-wiarygodność. **Null deviance** — przy modelu tylko z interceptem; **residual deviance** — przy aktualnym modelu.

Deviance residual (Poisson): $r_i^D = \mathrm{sign}(y_i - \hat{\mu}_i)\sqrt{2\left[y_i \log \frac{y_i}{\hat{\mu}_i} - (y_i - \hat{\mu}_i)\right]}$ (z konwencją $0 \log 0 = 0$).

Pakietów używanych w analizie: `car`, `lmtest`, `corrplot`, `MASS`, `ordinal` (instalacja: `install.packages(c("car","lmtest",`

2 Zadanie 1 — Regresja liniowa i interakcje

Źródła danych (zgodnie z poleceniem): (1.1) **Real Estate Valuation** (UCI ID 477) — `data/real_estate_uci.csv` (mirror CSV; przy błędzie 502 na `archive.ics.uci.edu` plik jest kopią tego samego zbioru). Zmienna zależna: `price`. (1.2) **Auto MPG** (UCI) — `data/auto_mpg_uci.csv`. Zmienna zależna: `mpg` (w treści polecenia błędnie napisano „niezależna"; w regresji `mpg` jest typową odpowiedzią).

2.1 1.1 Real Estate Valuation (UCI)

```
re_path <- file.path(proj_dir, "data", "real_estate_uci.csv")
stopifnot(file.exists(re_path))
re <- read.csv(re_path, stringsAsFactors = FALSE)
names(re) <- c("txn_date", "house_age", "dist_mrt", "n_stores", "lat", "lon", "price")
str(re)
```

```
## 'data.frame':    414 obs. of  7 variables:
##  $ txn_date : num  2013 2013 2014 2014 2013 ...
##  $ house_age: num  32 19.5 13.3 13.3 5 7.1 34.5 20.3 31.7 17.9 ...
##  $ dist_mrt : num  84.9 306.6 562 562 390.6 ...
##  $ n_stores : int  10 9 5 5 5 3 7 6 1 3 ...
##  $ lat      : num  25 25 25 25 25 ...
##  $ lon      : num  122 122 122 122 122 ...
##  $ price    : num  37.9 42.2 47.3 54.8 43.1 32.1 40.3 46.7 18.8 22.1 ...
```

```
sum(is.na(re))
```

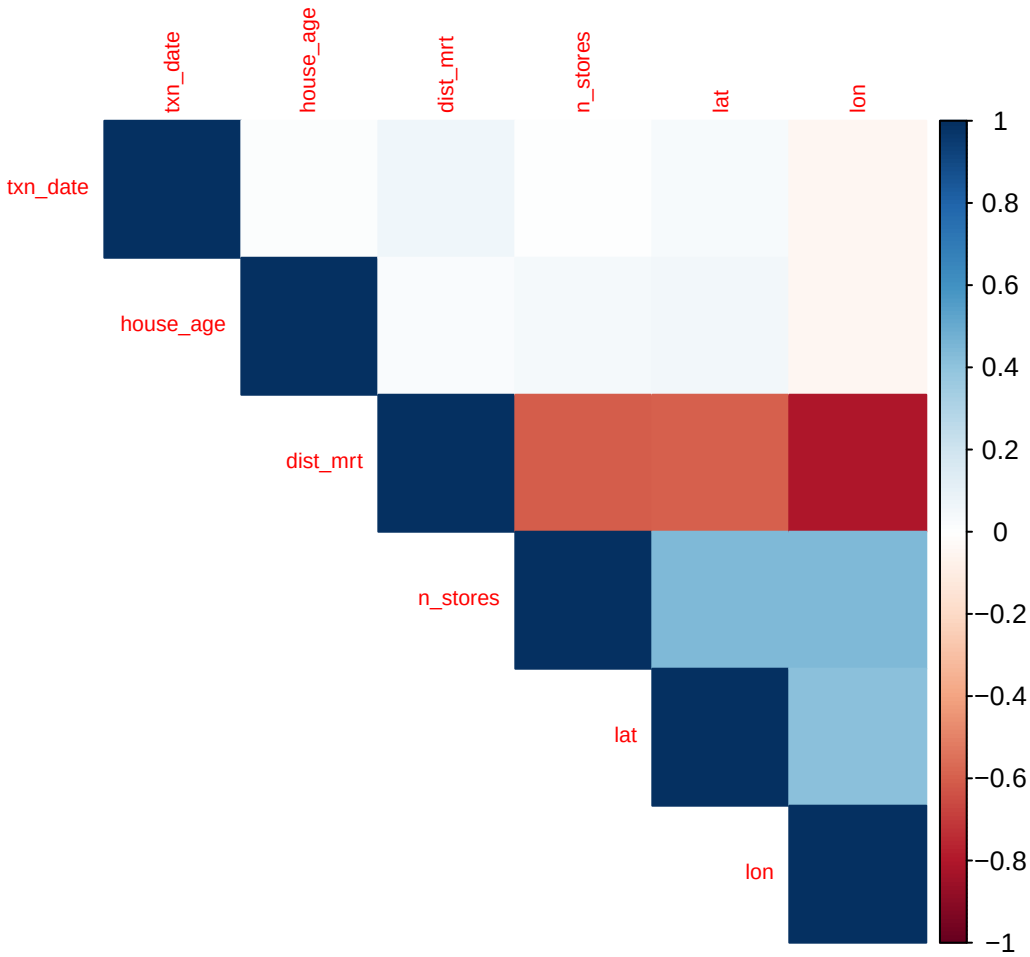
```
## [1] 0
```

(a) Zbiór UCI nie zawiera osobnej kolumny identyfikatora w pliku CSV — wszystkie kolumny poza `price` są numeryczne i nadają się do regresji liniowej (data transakcji `txn_date` jest kodowana numerycznie w oryginale).

(b) Braki: `sum(is.na(re))`. Przy brakach można stosować `na.omit`, imputację lub modele z obsługą `NA`.

(c) Macierz korelacji (Pearson), VIF oraz **test korelacji** dla przykładowej pary predyktorów:

```
num_re <- re[, setdiff(names(re), "price")]
M_re <- cor(num_re)
if (requireNamespace("corrplot", quietly = TRUE)) {
  corrplot::corrplot(M_re, method = "color", type = "upper", tl.cex = 0.7)
}
```




```r
fit_re_add <- lm(price ~ ., data = re)
car::vif(fit_re_add)
```

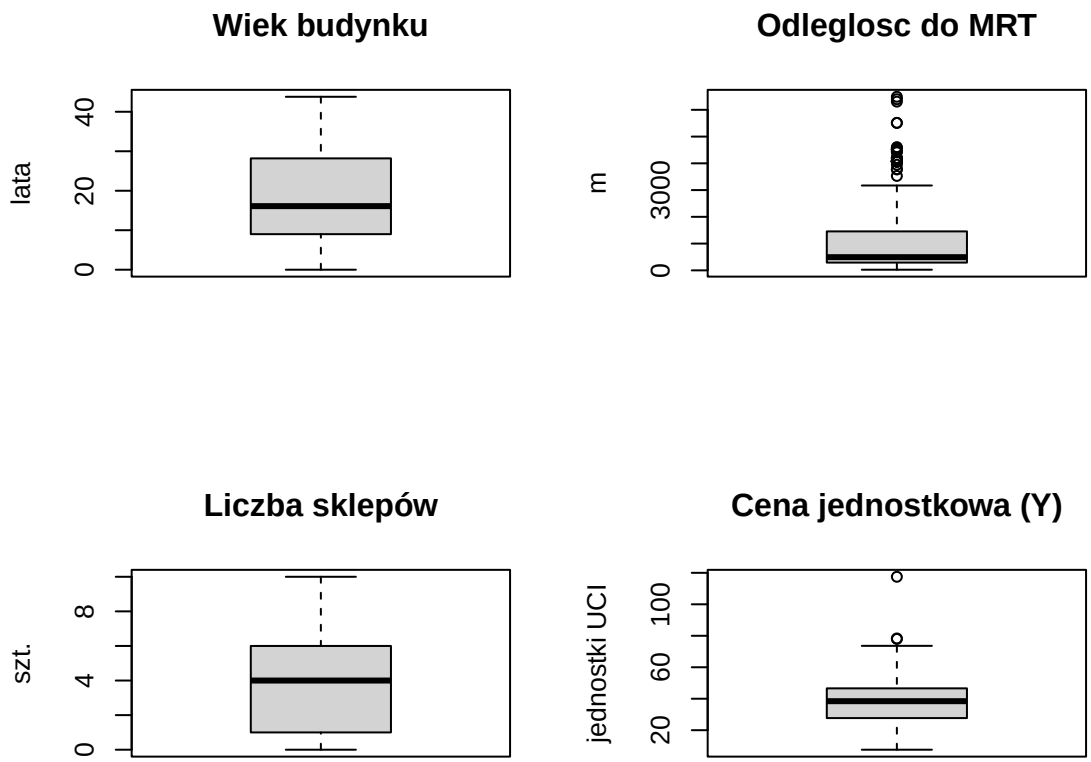
```
##  txn_date house_age  dist_mrt  n_stores       lat       lon
##  1.014674  1.014287  4.323019  1.617038  1.610234  2.926302
```

```r
cor.test(re$house_age, re$dist_mrt, method = "pearson")
```

```
##
##  Pearson's product-moment correlation
##
## data:  re$house_age and re$dist_mrt
## t = 0.52024, df = 412, p-value = 0.6032
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.07093092  0.12169934
## sample estimates:
##        cor
## 0.02562205
```

(d) Wizualizacja wartości odstających — boxploty wybranych zmiennych (duże odległości do MRT i ceny są szczególnie narażone na obserwacje skrajne). Możliwe reakcje: transformacja (np. log zmiennej silnie skośnej), winsoryzacja, modele odporne, usunięcie tylko po uzasadnieniu merytorycznym.

```r
par(mfrow = c(2, 2))
boxplot(re$house_age, main = "Wiek budynku", ylab = "lata")
boxplot(re$dist_mrt, main = "Odległość do MRT", ylab = "m")
boxplot(re$n_stores, main = "Liczba sklepów", ylab = "szt.")
boxplot(re$price, main = "Cena jednostkowa (Y)", ylab = "jednostki UCI")
```




```r
par(mfrow = c(1, 1))
```

Model z interakcjami — pełny kwadrat sześciu predyktorów daje bardzo wiele wyrazów; stosujemy kwadrat wybranego podzbioru silnie związanego z ceną (jak w typowej analizie tego zbioru):

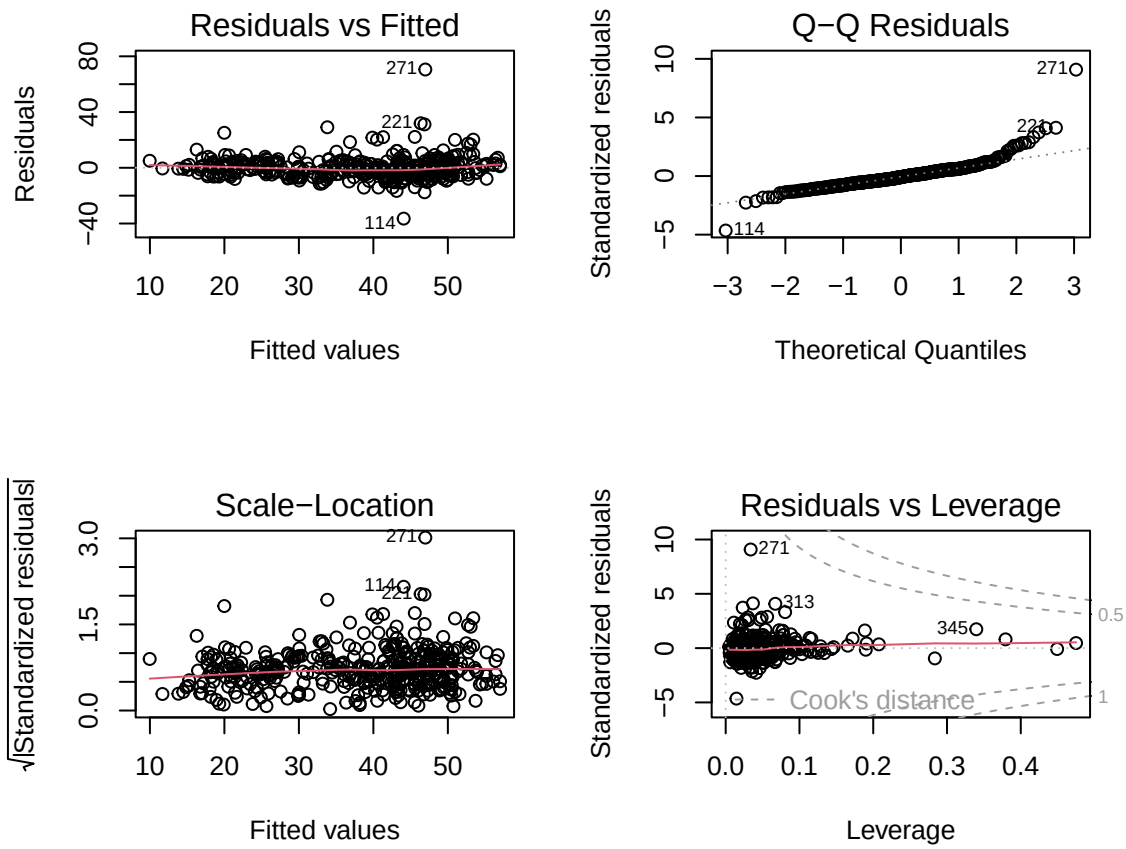
```r
fit_re <- lm(
  price ~ (house_age + dist_mrt + n_stores + lat + lon + txn_date)^2,
  data = re
)
summary(fit_re)
```

```
##
## Call:
## lm(formula = price ~ (house_age + dist_mrt + n_stores + lat +
##     lon + txn_date)^2, data = re)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
```

```
## -36.471  -4.328  -0.738   3.290  70.526
##
## Coefficients: (3 not defined because of singularities)
##                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)        -5.039e+04  1.724e+04  -2.922  0.00367 **
## house_age           3.325e+02  7.528e+02   0.442  0.65895
## dist_mrt            8.762e+00  4.172e+00   2.100  0.03634 *
## n_stores            5.420e+03  4.136e+03   1.311  0.19077
## lat                 9.939e+02  1.631e+02   6.092 2.65e-09 ***
## lon                 9.853e+01  1.178e+02   0.837  0.40337
## txn_date            6.777e+00  4.415e+00   1.535  0.12560
## house_age:dist_mrt  8.612e-06  7.877e-05   0.109  0.91300
## house_age:n_stores  9.445e-03  1.384e-02   0.682  0.49552
## house_age:lat      -2.897e+00  4.374e+00  -0.662  0.50819
## house_age:lon      -3.869e+00  5.762e+00  -0.671  0.50233
## house_age:txn_date  1.042e-01  1.260e-01   0.827  0.40879
## dist_mrt:n_stores  -1.687e-03  3.852e-04  -4.378 1.54e-05 ***
## dist_mrt:lat       -2.229e-01  4.396e-02  -5.072 6.08e-07 ***
## dist_mrt:lon       -1.212e-02  2.702e-02  -0.449  0.65393
## dist_mrt:txn_date  -8.575e-04  1.352e-03  -0.634  0.52619
## n_stores:lat       -1.071e+02  2.150e+01  -4.981 9.49e-07 ***
## n_stores:lon       -1.615e+01  3.119e+01  -0.518  0.60474
## n_stores:txn_date  -3.882e-01  6.075e-01  -0.639  0.52321
## lat:lon                    NA         NA      NA       NA
## lat:txn_date               NA         NA      NA       NA
## lon:txn_date               NA         NA      NA       NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.903 on 395 degrees of freedom
## Multiple R-squared:  0.6773, Adjusted R-squared:  0.6626
## F-statistic: 46.06 on 18 and 395 DF,  p-value: < 2.2e-16
```

Założenia (po `fit_re`):

```r
par(mfrow = c(2, 2))
plot(fit_re)
```




```r
par(mfrow = c(1, 1))
```

- **I) Liniowość:** reszty vs dopasowane (`plot(fit_re)`).
- **II) Autokorelacja:** `car::durbinWatsonTest(fit_re)` — dla danych przekrojowych kolejność jest umowna.
- **III) Homoskedastyczność:** `lmtest::bptest(fit_re)`.
- **IV) Normalność reszt:** QQ-plot; `shapiro.test` — przy dużym n mało czuły.

```r
car::durbinWatsonTest(fit_re)
```

```
##  lag Autocorrelation D-W Statistic p-value
##    1      -0.1358035       2.26588       0
##  Alternative hypothesis: rho != 0
```

```r
lmtest::bptest(fit_re)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  fit_re
## BP = 15.82, df = 18, p-value = 0.6051
```

```r
shapiro.test(residuals(fit_re))
```

```
##
##  Shapiro-Wilk normality test
```

```
##
## data:  residuals(fit_re)
## W = 0.83397, p-value < 2.2e-16
```

2.2 1.2 Auto MPG (UCI; zmienna zależna: `mpg`)

```r
auto_path <- file.path(proj_dir, "data", "auto_mpg_uci.csv")
stopifnot(file.exists(auto_path))
mpg <- read.csv(auto_path, check.names = TRUE)
names(mpg)[names(mpg) == "model.year"] <- "model_year"
mpg$horsepower <- suppressWarnings(as.numeric(as.character(mpg$horsepower)))
mpg <- na.omit(mpg)
str(mpg)
```

```
## 'data.frame':    396 obs. of  7 variables:
##  $ mpg         : num  18 15 18 16 17 15 14 14 14 15 ...
##  $ cylinders   : int  8 8 8 8 8 8 8 8 8 8 ...
##  $ displacement: num  307 350 318 304 302 429 454 440 455 390 ...
##  $ horsepower  : num  130 165 150 150 140 198 220 215 225 190 ...
##  $ weight      : int  3504 3693 3436 3433 3449 4341 4354 4312 4425 3850 ...
##  $ acceleration: num  12 11.5 11 12 10.5 10 9 8.5 10 8.5 ...
##  $ model_year  : int  70 70 70 70 70 70 70 70 70 70 ...
##  - attr(*, "na.action")= 'omit' Named int [1:2] 33 127
##   ..- attr(*, "names")= chr [1:2] "33" "127"
```

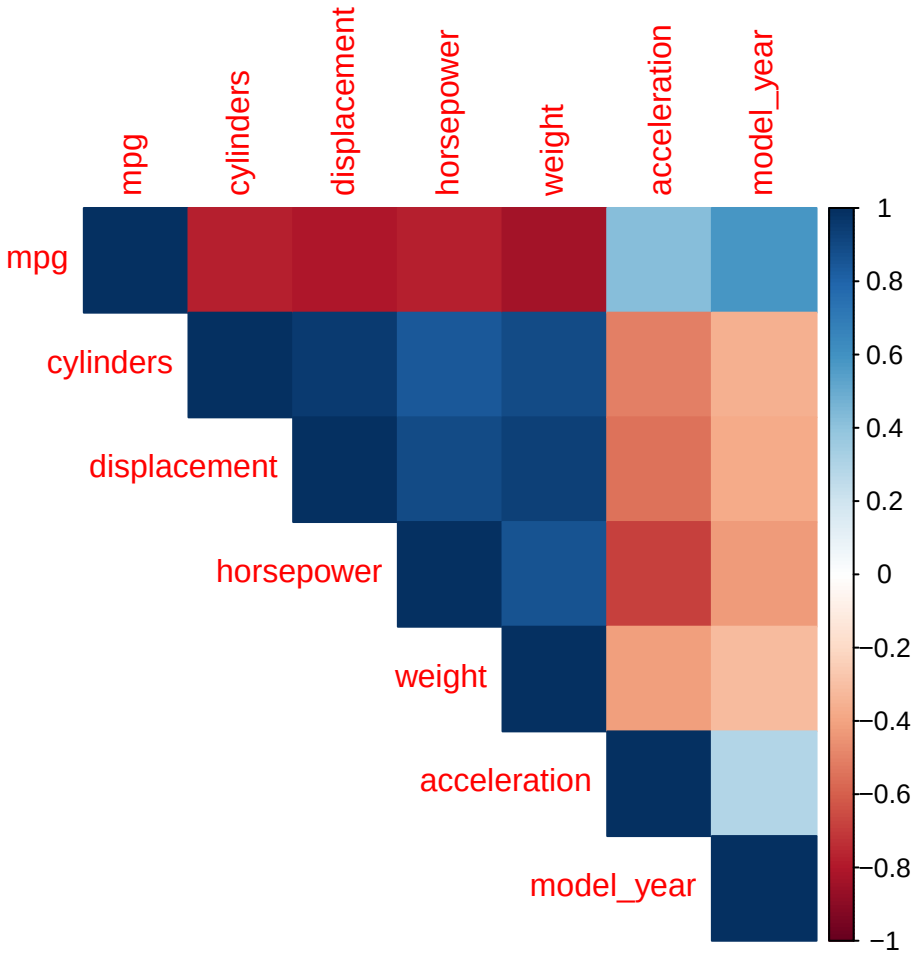
```r
sum(is.na(mpg))
```

```
## [1] 0
```

(a) Wykluczamy ewentualne kolumny tekstowe (w tym pliku ich nie ma). Wszystkie predyktory są numeryczne.

(b) `na.omit` po konwersji `horsepower` (w klasycznym pliku UCI czasem występują `?`).

```r
fit_m_add <- lm(mpg ~ ., data = mpg)
if (requireNamespace("corrplot", quietly = TRUE)) {
  num_mpg <- mpg[sapply(mpg, is.numeric)]
  corrplot::corrplot(cor(num_mpg), method = "color", type = "upper")
}
```




```r
car::vif(fit_m_add)
```

```
##    cylinders displacement   horsepower       weight acceleration   model_year
##    10.580334    19.722141     9.351519    10.622280     2.598238     1.251959
```

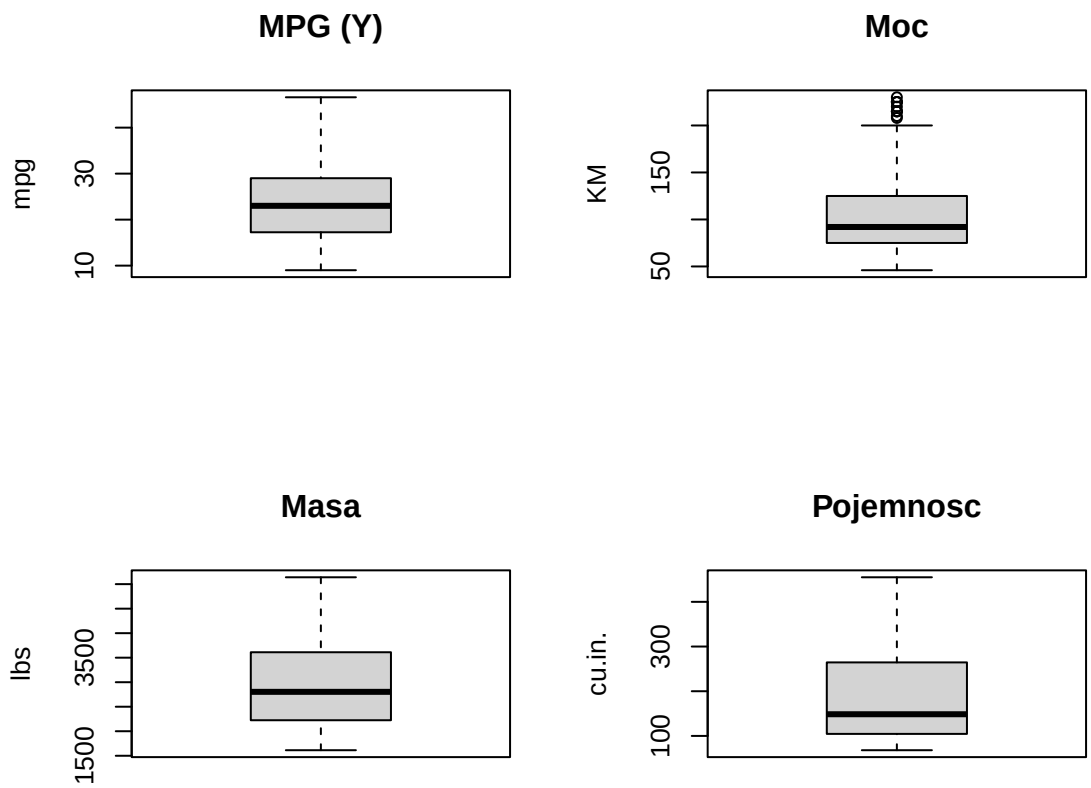
```r
cor.test(mpg$displacement, mpg$weight, method = "pearson")
```

```
##
##  Pearson's product-moment correlation
##
## data:  mpg$displacement and mpg$weight
## t = 51.346, df = 394, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.9186194 0.9444630
## sample estimates:
##       cor
## 0.9327289
```

(d) Boxploty pod kątem obserwacji odstających:

```r
par(mfrow = c(2, 2))
boxplot(mpg$mpg, main = "MPG (Y)", ylab = "mpg")
boxplot(mpg$horsepower, main = "Moc", ylab = "KM")
boxplot(mpg$weight, main = "Masa", ylab = "lbs")
```

```r
boxplot(mpg$displacement, main = "Pojemność", ylab = "cu.in.")
```



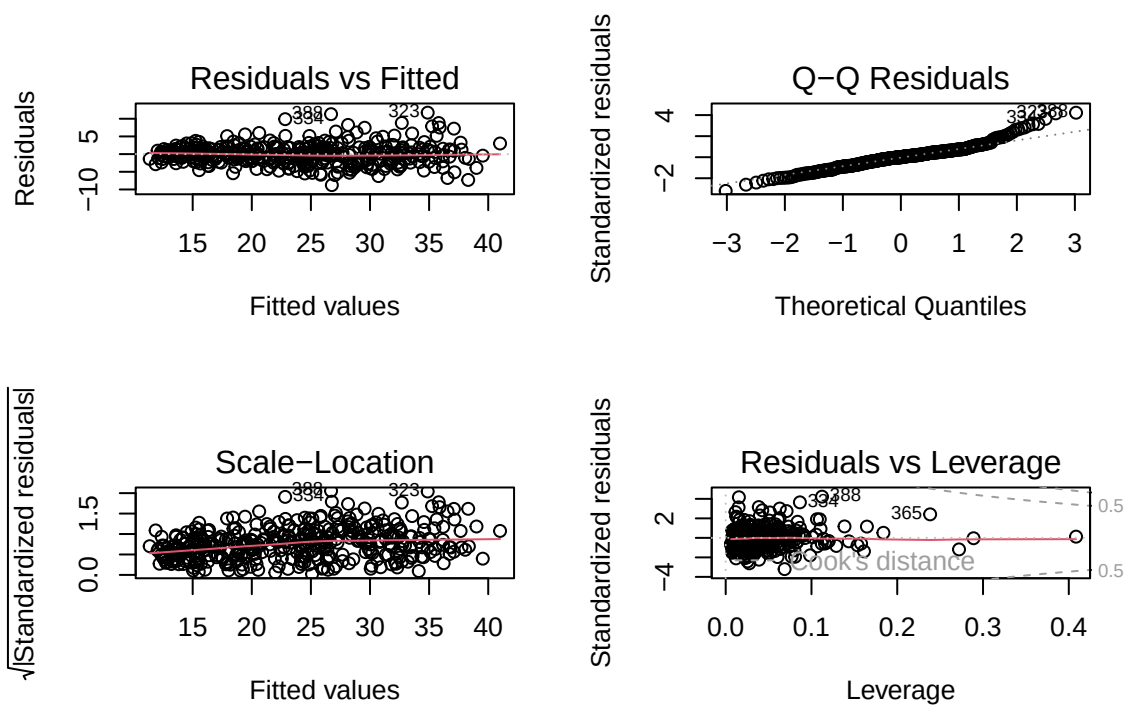

```r
par(mfrow = c(1, 1))
```

```r
fit_m <- lm(
  mpg ~ (displacement + weight + model_year + cylinders + acceleration)^2,
  data = mpg
)
summary(fit_m)
```

```
##
## Call:
## lm(formula = mpg ~ (displacement + weight + model_year + cylinders +
##     acceleration)^2, data = mpg)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.7724 -1.6457 -0.0069  1.3965 11.7194
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                 2.676e+01  3.645e+01   0.734  0.46328
## displacement               -2.251e-01  1.503e-01  -1.497  0.13523
```

```
## weight                     6.383e-03  1.359e-02   0.470  0.63896
## model_year                 5.117e-01  4.559e-01   1.122  0.26243
## cylinders                  7.167e+00  7.127e+00   1.006  0.31519
## acceleration              -6.544e+00  1.528e+00  -4.283 2.34e-05 ***
## displacement:weight        2.081e-05  6.999e-06   2.973  0.00314 **
## displacement:model_year    3.795e-03  2.058e-03   1.844  0.06593 .
## displacement:cylinders    -8.472e-03  3.678e-03  -2.304  0.02178 *
## displacement:acceleration -5.951e-03  2.391e-03  -2.489  0.01323 *
## weight:model_year         -3.476e-04  1.716e-04  -2.026  0.04348 *
## weight:cylinders           6.533e-04  5.850e-04   1.117  0.26478
## weight:acceleration        3.323e-04  1.892e-04   1.756  0.07995 .
## model_year:cylinders      -1.234e-01  9.024e-02  -1.367  0.17245
## model_year:acceleration    7.863e-02  1.915e-02   4.106 4.94e-05 ***
## cylinders:acceleration     1.567e-01  1.100e-01   1.425  0.15503
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.837 on 380 degrees of freedom
## Multiple R-squared:  0.8739, Adjusted R-squared:  0.8689
## F-statistic: 175.5 on 15 and 380 DF,  p-value: < 2.2e-16
```

```r
par(mfrow = c(2, 2))
plot(fit_m)
```




```r
par(mfrow = c(1, 1))
car::durbinWatsonTest(fit_m)
```

```
##  lag Autocorrelation D-W Statistic p-value
##    1       0.2658752      1.465105       0
##  Alternative hypothesis: rho != 0
```

```r
lmtest::bptest(fit_m)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  fit_m
## BP = 44.013, df = 15, p-value = 0.0001095
```

```r
shapiro.test(residuals(fit_m))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(fit_m)
## W = 0.96374, p-value = 2.505e-08
```

---

3 Zadanie 2 — Regresja Poissona (`poisson.data`)

Log-wiarygodność (niezależne $Y_i \sim \text{Poisson}(\lambda_i)$, $\log \lambda_i = a_0 + a_1 x_i$):

$$\ell(a_0, a_1) = \sum_{i=1}^{n} \left[y_i(a_0 + a_1 x_i) - e^{a_0 + a_1 x_i} - \log(y_i!) \right].$$

Maksymalizacja ℓ jest równoważna minimalizacji **ujemnej** log-wiarygodności bez stałego członu $\log(y_i!)$.

```r
poisson_df <- read.csv2(
  file.path(proj_dir, "poisson.data"),
  dec = ",",
  sep = ";",
  stringsAsFactors = FALSE
)
names(poisson_df) <- gsub('"', "", names(poisson_df))
poisson_df$id <- NULL
poisson_df$X1 <- as.numeric(gsub(",", ".", poisson_df$X1))
poisson_df$Y <- as.integer(poisson_df$Y)
str(poisson_df)
```

```
## 'data.frame':    100 obs. of  2 variables:
##  $ X1: num  8.75 4.78 9.53 6.29 1.91 ...
##  $ Y : int  212 33 299 68 8 287 13 43 4 4 ...
```

```r
nll_poisson <- function(par, x, y) {
  eta <- par[1] + par[2] * x
  mu <- exp(eta)
  -sum(dpois(y, lambda = mu, log = TRUE))
}
opt <- optim(
  par = c(0, 0),
  fn = nll_poisson,
  x = poisson_df$X1,
  y = poisson_df$Y,
  method = "BFGS"
)
opt$par
```

```
## [1] 1.0332622 0.4975154
```

```r
fit_p <- glm(Y ~ X1, data = poisson_df, family = poisson())
summary(fit_p)
```

```
##
## Call:
## glm(formula = Y ~ X1, family = poisson(), data = poisson_df)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) 1.032997   0.051017   20.25   <2e-16 ***
## X1          0.497547   0.006194   80.33   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 9945.17  on 99  degrees of freedom
## Residual deviance:  101.89  on 98  degrees of freedom
## AIC: 662.87
##
## Number of Fisher Scoring iterations: 4
```

Porównanie estymat `optim` (minimalizacja NLL) i `glm` (maksymalizacja tej samej log-wiarygodności):

```r
data.frame(
  parametr = c("a0 (intercept)", "a1 (X1)"),
  optim = opt$par,
  glm = coef(fit_p)
)
```

```
##                   parametr     optim       glm
## (Intercept) a0 (intercept) 1.0332622 1.0329973
## X1                 a1 (X1) 0.4975154 0.4975466
```

Odpowiedzi (2.6):

- **(a)** Deviance residuals mierzą odległość punktu od dopasowania w skali deviance; przy dobrym dopasowaniu powinny być w przybliżeniu symetryczne wokół 0 (niekoniecznie normalne).
- **(b)** Estymaty `optim` i `glm` powinny być bliskie (ten sam model); różnice to efekt optymalizacji numerycznej.
- **(c)** Null deviance odpowiada modelowi z samym interceptem; residual deviance — modelowi z `X1`. Ich różnica (przy rozsądnych założeniach) może służyć do testu istotności bloku predyktorów (^{2} o odpowiedniej liczbie stopni swobody).

---

4 Zadanie 3 — `ships` (MASS)

```r
data("ships", package = "MASS")
ships$type <- factor(ships$type)
ships$year <- factor(ships$year)
ships$period <- factor(ships$period)
ships <- subset(ships, service > 0)
str(ships)
```

```
## 'data.frame':    34 obs. of  5 variables:
##  $ type     : Factor w/ 5 levels "A","B","C","D",..: 1 1 1 1 1 1 1 2 2 2 ...
##  $ year     : Factor w/ 4 levels "60","65","70",..: 1 1 2 2 3 3 4 1 1 2 ...
##  $ period   : Factor w/ 2 levels "60","75": 1 2 1 2 1 2 2 1 2 1 ...
##  $ service  : int  127 63 1095 1095 1512 3353 2244 44882 17176 28609 ...
##  $ incidents: int  0 0 3 4 6 18 11 39 29 58 ...
```

```r
summary(ships)
```

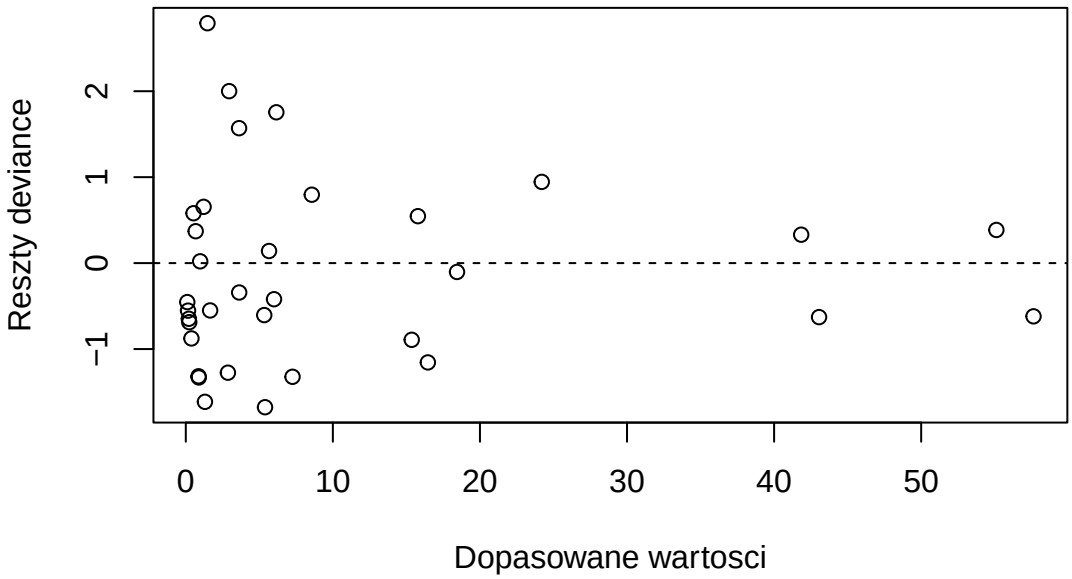
```
##  type  year    period     service        incidents
##  A:7   60: 9   60:15   Min.   :   45   Min.   : 0.00
##  B:7   65:10   75:19   1st Qu.:  371   1st Qu.: 1.00
##  C:7   70:10           Median : 1095   Median : 4.00
##  D:7   75: 5           Mean   : 4811   Mean   :10.47
##  E:6                   3rd Qu.: 2223   3rd Qu.:11.75
##                        Max.   :44882   Max.   :58.00
```

```r
fit_ships <- glm(
  incidents ~ type + year + period,
  data = ships,
  family = poisson,
  offset = log(service)
)
summary(fit_ships)
```

```
##
## Call:
## glm(formula = incidents ~ type + year + period, family = poisson,
##     data = ships, offset = log(service))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -6.40590    0.21744 -29.460  < 2e-16 ***
## typeB       -0.54334    0.17759  -3.060  0.00222 **
## typeC       -0.68740    0.32904  -2.089  0.03670 *
## typeD       -0.07596    0.29058  -0.261  0.79377
## typeE        0.32558    0.23588   1.380  0.16750
## year65       0.69714    0.14964   4.659 3.18e-06 ***
## year70       0.81843    0.16977   4.821 1.43e-06 ***
## year75       0.45343    0.23317   1.945  0.05182 .
## period75     0.38447    0.11827   3.251  0.00115 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 146.328  on 33  degrees of freedom
## Residual deviance:  38.695  on 25  degrees of freedom
## AIC: 154.56
##
## Number of Fisher Scoring iterations: 5
```

Interpretacja: `offset(log(service))` uwzględnia różną ekspozycję (miesiące). Wiersze z `service == 0` pominięto, bo log(0) nie jest określone (brak ekspozycji). Czynniki kodowane są kontrastem „treatment” (domyślnie w R pierwszy poziom = referencja).

```r
r <- residuals(fit_ships, type = "deviance")
plot(fitted(fit_ships), r, xlab = "Dopasowane wartości", ylab = "Reszty deviance")
abline(h = 0, lty = 2)
```



Normalność deviance residuals: to nie są klasyczne reszty OLS; oczekiwanie ścisłej normalności nie jest takim samym założeniem jak w regresji liniowej — oceniamy głównie losowość i brak systematycznych wzorów.

---

5 Zadanie 4 — `esoph`

```r
data("esoph", package = "datasets")
esoph <- subset(esoph, ncontrols > 0)
str(esoph)
```

```
## 'data.frame':    76 obs. of  5 variables:
##  $ agegp    : Ord.factor w/ 6 levels "25-34"<"35-44"<..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ alcgp    : Ord.factor w/ 4 levels "0-39g/day"<"40-79"<..: 1 1 1 1 2 2 2 2 3 3 ...
##  $ tobgp    : Ord.factor w/ 4 levels "0-9g/day"<"10-19"<..: 1 2 3 4 1 2 3 4 1 2 ...
##  $ ncases   : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ ncontrols: num  40 10 6 5 27 7 4 7 2 1 ...
```

```r
fit_e_full <- glm(
  ncases ~ agegp + alcgp + tobgp,
  data = esoph,
  family = poisson,
  offset = log(ncontrols)
)
```

```r
summary(fit_e_full)
```

```
##
## Call:
## glm(formula = ncases ~ agegp + alcgp + tobgp, family = poisson,
##     data = esoph, offset = log(ncontrols))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -3.9107   206.2276  -0.019   0.9849
## agegp.L      12.5592   739.4675   0.017   0.9864
## agegp.Q     -10.0607   675.0384  -0.015   0.9881
## agegp.C       5.6734   461.1390   0.012   0.9902
## agegp^4      -2.8307   233.8402  -0.012   0.9903
## agegp^5       0.7160    77.9469   0.009   0.9927
## alcgp.L       2.0460     0.1844  11.095  < 2e-16 ***
## alcgp.Q      -0.1111     0.1648  -0.674   0.5002
## alcgp.C       0.3626     0.1440   2.517   0.0118 *
## tobgp.L       0.9439     0.1826   5.169 2.36e-07 ***
## tobgp.Q       0.2494     0.1717   1.453   0.1463
## tobgp.C       0.2207     0.1612   1.369   0.1710
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 390.970  on 75  degrees of freedom
## Residual deviance:  89.255  on 64  degrees of freedom
## AIC: 254.54
##
## Number of Fisher Scoring iterations: 16
```

Uwaga: pominięto komórki z `ncontrols == 0`, bo offset log(`ncontrols`) nie jest określony.

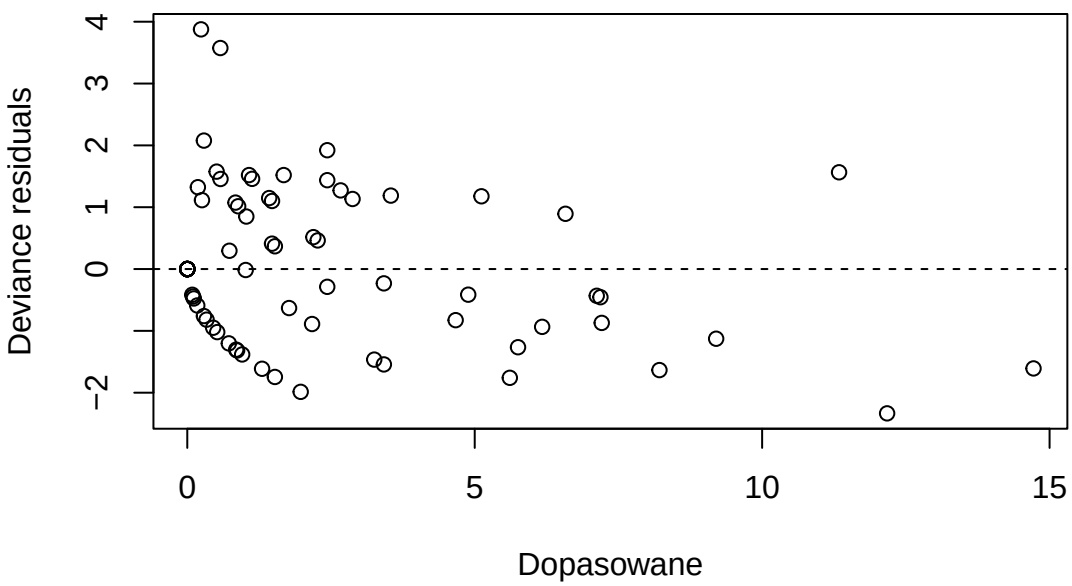
```r
fit_e_red <- glm(
  ncases ~ agegp + alcgp,
  data = esoph,
  family = poisson,
  offset = log(ncontrols)
)
summary(fit_e_red)
```

```
##
## Call:
## glm(formula = ncases ~ agegp + alcgp, family = poisson, data = esoph,
##     offset = log(ncontrols))
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -4.12041  214.17559  -0.019   0.9847
## agegp.L      12.37420  767.96633   0.016   0.9871
## agegp.Q      -9.94463  701.05413  -0.014   0.9887
## agegp.C       5.68800  478.91118   0.012   0.9905
## agegp^4      -2.76248  242.85237  -0.011   0.9909
## agegp^5       0.74250   80.95094   0.009   0.9927
```

```
## alcgp.L       2.11923    0.18305  11.577   <2e-16 ***
## alcgp.Q      -0.08877    0.16239  -0.547   0.5846
## alcgp.C       0.38488    0.14221   2.706   0.0068 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 390.97  on 75  degrees of freedom
## Residual deviance: 113.40  on 67  degrees of freedom
## AIC: 272.69
##
## Number of Fisher Scoring iterations: 16
```

Po usunięciu nieistotnego `tobgp` estymaty pozostałych mogą się nieznacznie zmienić (korelacja / kontrast z innymi czynnikami).

```r
r4 <- residuals(fit_e_red, type = "deviance")
plot(fitted(fit_e_red), r4, xlab = "Dopasowane", ylab = "Deviance residuals")
abline(h = 0, lty = 2)
```



---

6 Zadanie 5 — CLM (`clm.data`)

```r
clm_df <- read.csv(file.path(proj_dir, "clm.data"), stringsAsFactors = FALSE)
clm_df$id <- NULL
clm_df$satysfakcja <- factor(clm_df$satysfakcja, ordered = TRUE)
str(clm_df)
```

```
## 'data.frame':    100 obs. of  3 variables:
##  $ satysfakcja: Ord.factor w/ 5 levels "1"<"2"<"3"<"4"<..: 3 3 2 2 3 5 4 1 2 3 ...
##  $ cena       : num  96.9 156.2 89.8 139.2 122.2 ...
##  $ obsluga    : int  10 7 9 7 10 4 8 9 9 9 ...
```

Istotne atrybuty: `cena` i `obsluga` jako predyktory ciągłe / numeryczne wpływu na próg satysfakcji.

```r
library(ordinal)
fit_clm <- clm(satysfakcja ~ cena + obsluga, data = clm_df)
summary(fit_clm)
```

```
## formula: satysfakcja ~ cena + obsluga
## data:    clm_df
##
##  link  threshold nobs logLik  AIC    niter max.grad cond.H
##  logit flexible  100  -159.66 331.31 4(0)  1.69e-12 1.2e+06
##
## Coefficients:
##          Estimate Std. Error z value Pr(>|z|)
## cena    -0.005178   0.004391  -1.179    0.238
## obsluga  0.014197   0.058268   0.244    0.808
##
## Threshold coefficients:
##      Estimate Std. Error z value
## 1|2 -1.926818   0.748672  -2.574
## 2|3 -0.953354   0.730469  -1.305
## 3|4 -0.001363   0.719511  -0.002
## 4|5  0.879775   0.725654   1.212
```

Interpretacja: szacowane **progi** dzielą skalę latentną na kategorie; współczynniki `cena`, `obsluga` to efekty na logitach prawdopodobieństw kumulatywnych (przy założeniu proporcjonalnych szans).

---

7 Zadanie 6 — `wine` (ordinal)

```r
data(wine, package = "ordinal")
str(wine)
```

```
## 'data.frame':    72 obs. of  6 variables:
##  $ response: num  36 48 47 67 77 60 83 90 17 22 ...
##  $ rating  : Ord.factor w/ 5 levels "1"<"2"<"3"<"4"<..: 2 3 3 4 4 4 5 5 1 2 ...
##  $ temp    : Factor w/ 2 levels "cold","warm": 1 1 1 1 2 2 2 2 1 1 ...
##  $ contact : Factor w/ 2 levels "no","yes": 1 1 2 2 1 1 2 2 1 1 ...
##  $ bottle  : Factor w/ 8 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 1 2 ...
##  $ judge   : Factor w/ 9 levels "1","2","3","4",..: 1 1 1 1 1 1 1 1 2 2 ...
```

```r
fit_w <- clm(rating ~ temp + contact, data = wine)
summary(fit_w)
```

```
## formula: rating ~ temp + contact
## data:    wine
##
##  link  threshold nobs logLik AIC    niter max.grad cond.H
##  logit flexible  72   -86.49 184.98 6(0)  4.02e-12 2.7e+01
##
```

```
## Coefficients:
##            Estimate Std. Error z value Pr(>|z|)
## tempwarm     2.5031     0.5287   4.735 2.19e-06 ***
## contactyes   1.5278     0.4766   3.205  0.00135 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Threshold coefficients:
##     Estimate Std. Error z value
## 1|2  -1.3444     0.5171  -2.600
## 2|3   1.2508     0.4379   2.857
## 3|4   3.4669     0.5978   5.800
## 4|5   5.0064     0.7309   6.850
```

```r
pr <- predict(fit_w, type = "prob")
head(pr$fit, 3)
```

```
## [1] 0.5706497 0.1922909 0.4430599
```

`predict(..., type = "prob")` zwraca listę; macierz `fit` zawiera prawdopodobieństwa klas dla każdej obserwacji.

```r
rating_num <- as.integer(as.character(wine$rating))
wine$rating_bin <- factor(
  ifelse(rating_num < 4, 0, 1),
  levels = c(0, 1),
  ordered = TRUE
)
wine$y_high <- as.integer(rating_num >= 4)
fit_w_bin_clm <- clm(rating_bin ~ temp + contact, data = wine)
fit_w_bin_glm <- glm(y_high ~ temp + contact, data = wine, family = binomial())
summary(fit_w_bin_clm)
```

```
## formula: rating_bin ~ temp + contact
## data:    wine
##
##  link  threshold nobs logLik AIC   niter max.grad cond.H
##  logit flexible  72   -28.73 63.46 6(0)  3.21e-12 3.1e+01
##
## Coefficients:
##            Estimate Std. Error z value Pr(>|z|)
## tempwarm     3.0312     0.8525   3.556 0.000377 ***
## contactyes   1.8102     0.6927   2.613 0.008964 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Threshold coefficients:
##     Estimate Std. Error z value
## 0|1   4.0719     0.9393   4.335
```

```r
summary(fit_w_bin_glm)
```

```
##
## Call:
## glm(formula = y_high ~ temp + contact, family = binomial(), data = wine)
##
## Coefficients:
```

```
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -4.0719     0.9392  -4.335 1.45e-05 ***
## tempwarm      3.0312     0.8525   3.556 0.000377 ***
## contactyes    1.8102     0.6927   2.613 0.008963 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 83.100  on 71  degrees of freedom
## Residual deviance: 57.457  on 69  degrees of freedom
## AIC: 63.457
##
## Number of Fisher Scoring iterations: 5
```

Przy **dwóch** poziomach porządkowych CLM (logit proporcjonalnych szans) i logistyka opisują podobny efekt skokowy; różnią się parametryzacją progów vs intercept logistyczny, ale kierunek i istotność predyktorów powinny być spójne.

---

8 Podsumowanie

W sprawozdaniu wykonano analizy zgodnie z `polecenie.txt`: regresję liniową z interakcjami na zbiorach Real Estate (UCI) i Auto MPG (pliki w `data/`), GLM Poissona (własna optymalizacja i `glm`), modele dla `ships` i `esoph`, modele porządkowe CLM oraz porównanie z logistyką po binarnej recode `rating`. Przed oddaniem uzupełnij pole `author:` w nagłówku YAML.